

MECHATRONICS AND IOT

ASSIGNMENT 1

MQTT-BASED REMOTE HVAC MONITORING SYSTEM USING ADAFRUIT CLOUD

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

MQTT-BASED REMOTE HVAC MONITORING SYSTEM USING ADAFRUIT CLOUD

1. Project Overview

The MQTT-Based Remote HVAC Monitoring System is an IoT-based system designed to monitor important environmental parameters of an HVAC (Heating, Ventilation and Air Conditioning) system remotely. The system uses an ESP8266 Node MCU as the main controller and a DHT11 temperature and humidity sensor to measure the surrounding environmental conditions.

The measured data is displayed locally on an I2C LCD display and transmitted to the Adafruit cloud platform through the Internet using the MQTT communication protocol. The cloud platform stores the collected data, allowing users to monitor and analyse temperature and humidity variations remotely.

The system provides a low-cost approach for HVAC monitoring and can be further developed to include automatic control of fans, air conditioners and other HVAC equipment.

2. Project Statement

Traditional HVAC systems generally require local monitoring of temperature and humidity conditions. Continuous manual monitoring can be inconvenient, especially when the system is installed in laboratories, classrooms, offices, industries or other large buildings.

Therefore, there is a need for a remote HVAC monitoring system that can:

- Measure temperature continuously.
- Measure relative humidity.
- Display parameters locally.
- Send sensor data to the cloud.
- Allow remote monitoring through the Internet.

- Store historical sensor data for analysis.
- Provide a foundation for future automatic HVAC control.

The proposed system addresses these requirements using ESP8266, MQTT and Adafruit Cloud.

3. Proposed Solution

The proposed solution consists of an ESP8266 Node MCU, DHT11 sensor, I2C LCD display and Adafruit cloud platform.

The DHT11 sensor measures temperature and humidity and sends the readings to the ESP8266. The ESP8266 processes the sensor data and displays the values on the LCD.

At the same time, the ESP8266 connects to a Wi-Fi network and publishes the sensor readings using the MQTT protocol. Adafruit receives and stores the data in the cloud.

Main functions

1. Temperature measurement.
2. Humidity measurement.
3. Local LCD display.
4. Wi-Fi connectivity.
5. MQTT-based communication.
6. Cloud data storage.
7. Remote monitoring.

Future enhancement

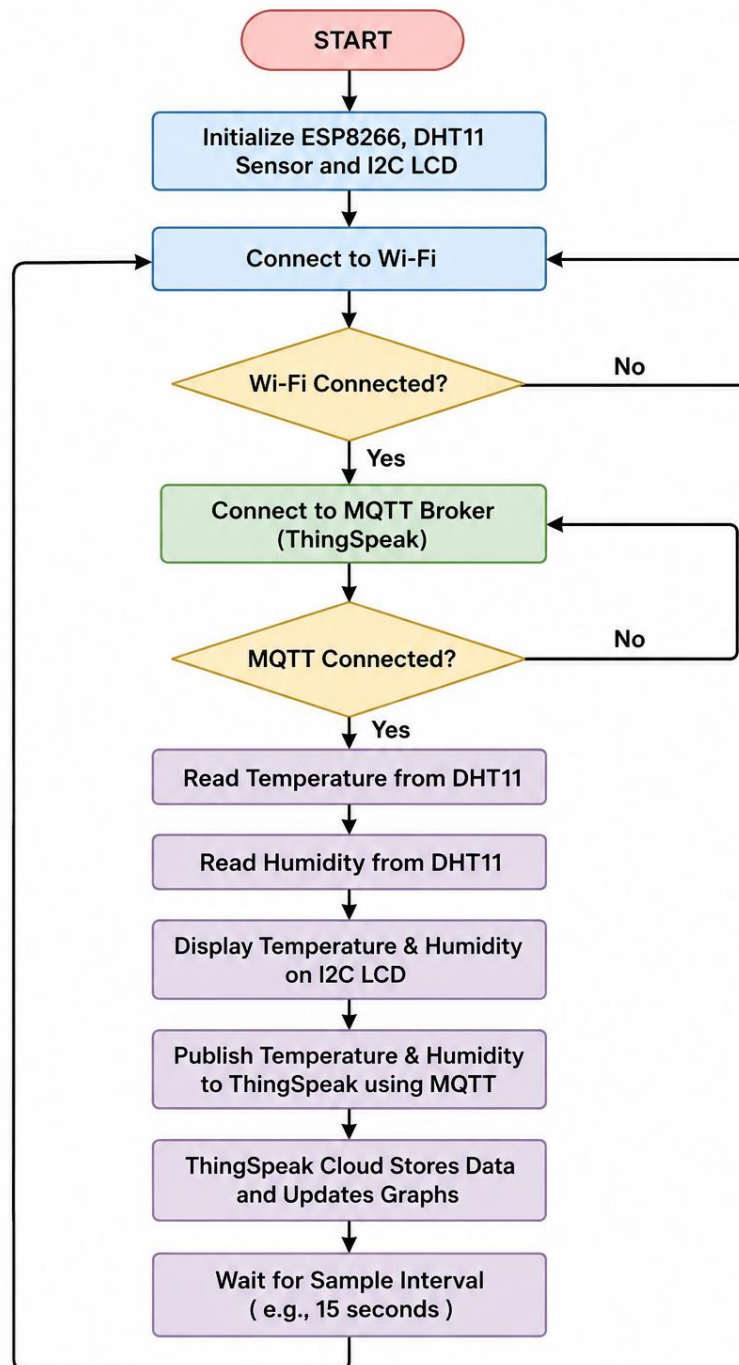
The system can also be extended to control:

- Exhaust fans
- Cooling fans

- Air conditioners
- Ventilation systems
- Relays and actuators

based on predefined temperature or humidity limits.

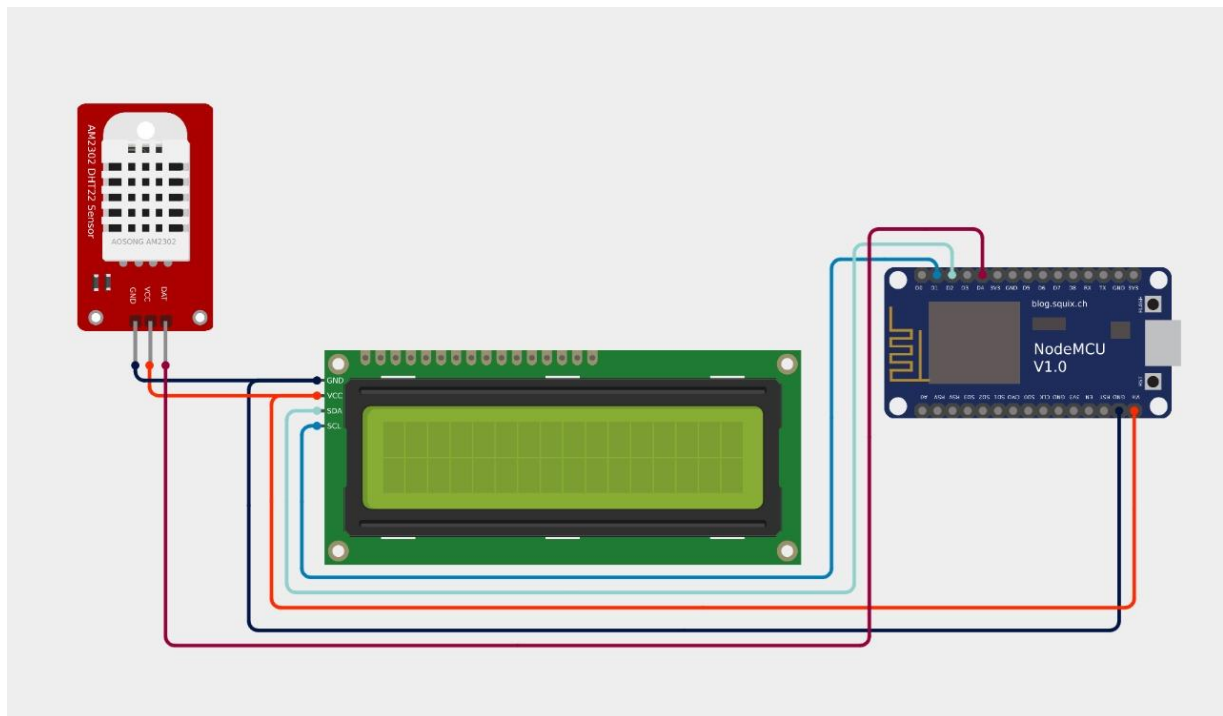
4. System Architecture:



5. Circuit Table:

Component	Pin	ESP8266 Pin	Function
DHT11	VCC	3.3V	Power supply
DHT11	GND	GND	Ground
DHT11	DATA	GPIO2 (D4)	Temperature & humidity data
I2C LCD	VCC	5V/VIN	Power
I2C LCD	GND	GND	Ground
I2C LCD	SDA	GPIO2 (D2)	I2C data
I2C LCD	SCL	GPIO2 (D1)	I2C clock
ESP8266	VIN/USB	5V USB	Power supply

6. Circuit Design:



7. Algorithm:

Step-by-step algorithm

Step 1: Start the system.

Step 2: Initialize the ESP8266.

Step 3: Initialize the DHT11 sensor.

Step 4: Initialize the I2C LCD.

Step 5: Connect the ESP8266 to the configured Wi-Fi network.

Step 6: Establish MQTT communication with the ThingSpeak MQTT server.

Step 7: Read temperature from the DHT11 sensor.

Step 8: Read relative humidity from the DHT11 sensor.

Step 9: Display temperature and humidity on the LCD.

Step 10: Publish the sensor values through MQTT.

Step 11: Adafruit receives and stores the data.

Step 12: Repeat the process at a predefined interval.

8. Coding:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>

// =====

// ADAFRUIT IO

// =====

#include "AdafruitIO_WiFi.h"

// WiFi#define WIFI_SSID    "OnePlus Nord CE5 7C5C"

#define WIFI_PASS    "qkts5737"

// Adafruit IO
```

```
#define IO_USERNAME  "Sivasaravanan"

#define IO_KEY      "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"

// Connect to Adafruit IO

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// Adafruit IO feeds

AdafruitIO_Feed *temperatureFeed = io.feed("temperature");

AdafruitIO_Feed *humidityFeed = io.feed("humidity");

// =====

// DHT11 SETTINGS

// =====

#define DHTPIN 2    // D4 / GPIO2

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// =====

// I2C LCD

// =====

LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====

// SETUP

// =====

void setup()

{

  Serial.begin(9600);

  // Start DHT11

  dht.begin();

  // ESP8266 I2C

  // SDA = D2 / GPIO4

  // SCL = D1 / GPIO5

  Wire.begin(4, 5);
```

```
// Start LCD
lcd.init();
lcd.backlight();

// Starting message
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("HVAC MONITOR");
lcd.setCursor(0, 1);
lcd.print("Starting...");
delay(2000);

// =====
// CONNECT TO ADAFRUIT IO
// =====

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("HVAC MONITOR");
lcd.setCursor(0, 1);
lcd.print("Connecting...");
Serial.println("Connecting to Adafruit IO...");
io.connect();
while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}
Serial.println();
Serial.println("Adafruit IO Connected!");
lcd.clear();
lcd.setCursor(0, 0);
```



```
lcd.print("HVAC MONITOR");

lcd.setCursor(0, 1);

lcd.print("IO Connected");

delay(2000);

lcd.clear();

}

// =====

// LOOP

// =====

void loop()

{

    // Keep Adafruit IO connection alive

    io.run();

    // Read DHT11

    float temperature = dht.readTemperature();

    float humidity = dht.readHumidity();

    // =====

    // CHECK SENSOR

    // =====

    if (isnan(temperature) || isnan(humidity))

    {

        Serial.println("DHT11 ERROR!");

        lcd.clear();

        lcd.setCursor(0, 0);

        lcd.print("DHT11 ERROR");

        lcd.setCursor(0, 1);

        lcd.print("Check Sensor");

        delay(2000);

        return;

    }

}
```

```
}  
  
// =====  
  
// SERIAL MONITOR  
  
// =====  
  
Serial.println("-----");  
  
Serial.print("Temperature: ");  
  
Serial.print(temperature);  
  
Serial.println(" C");  
  
Serial.print("Humidity: ");  
  
Serial.print(humidity);  
  
Serial.println(" %");  
  
// =====  
  
// SEND TO ADAFRUIT IO  
  
// =====  
  
temperatureFeed->save(temperature);  
  
humidityFeed->save(humidity);  
  
Serial.println("Data sent to Adafruit IO")  
  
// =====  
  
// LCD DISPLAY  
  
// =====  
  
lcd.clear();  
  
lcd.setCursor(0, 0);  
  
lcd.print("Temp: ");  
  
lcd.print(temperature, 1);  
  
lcd.write(223);  
  
lcd.print("C");  
  
lcd.setCursor(0, 1);  
  
lcd.print("Hum : ");  
  
lcd.print(humidity, 1);
```

```

lcd.print("%");

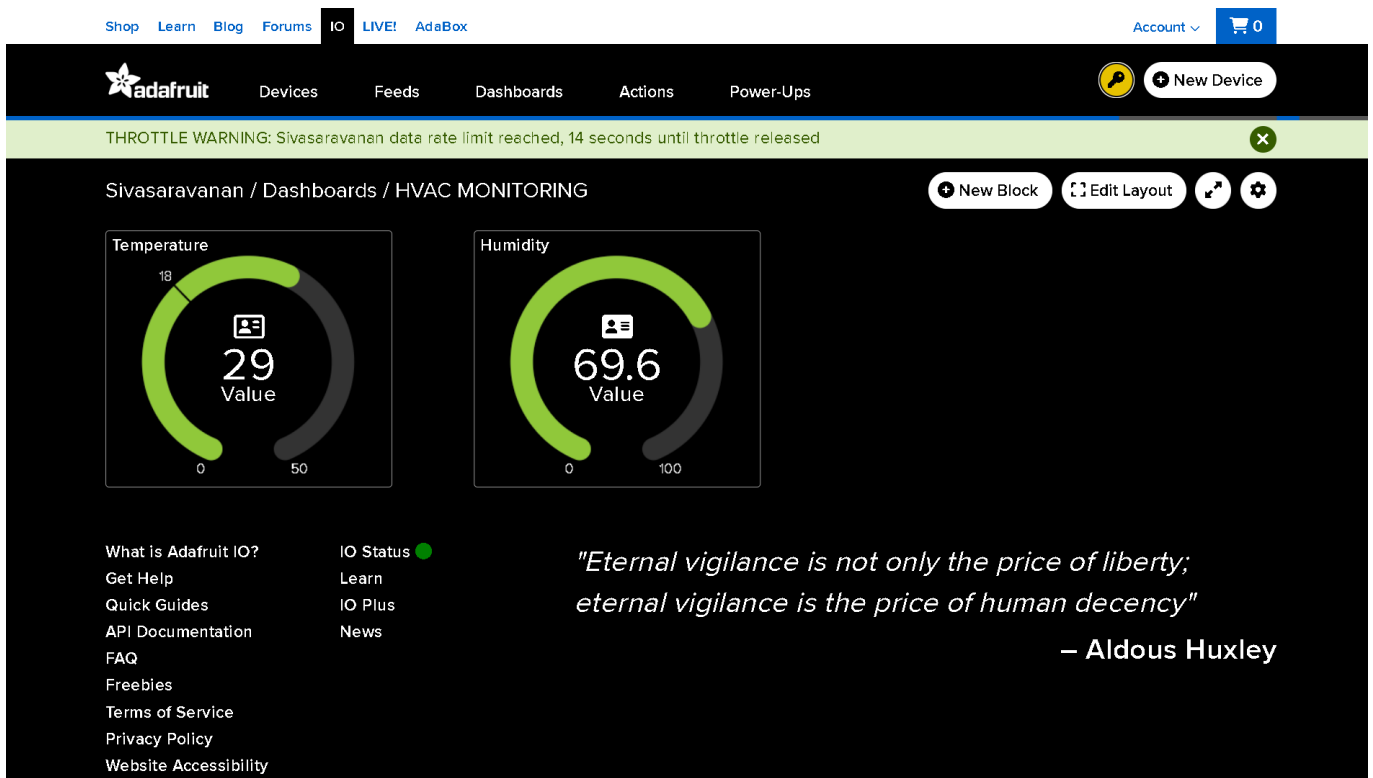
// =====
// READ EVERY 2 SECONDS
// =====

delay(2000);

}

```

9. System Working



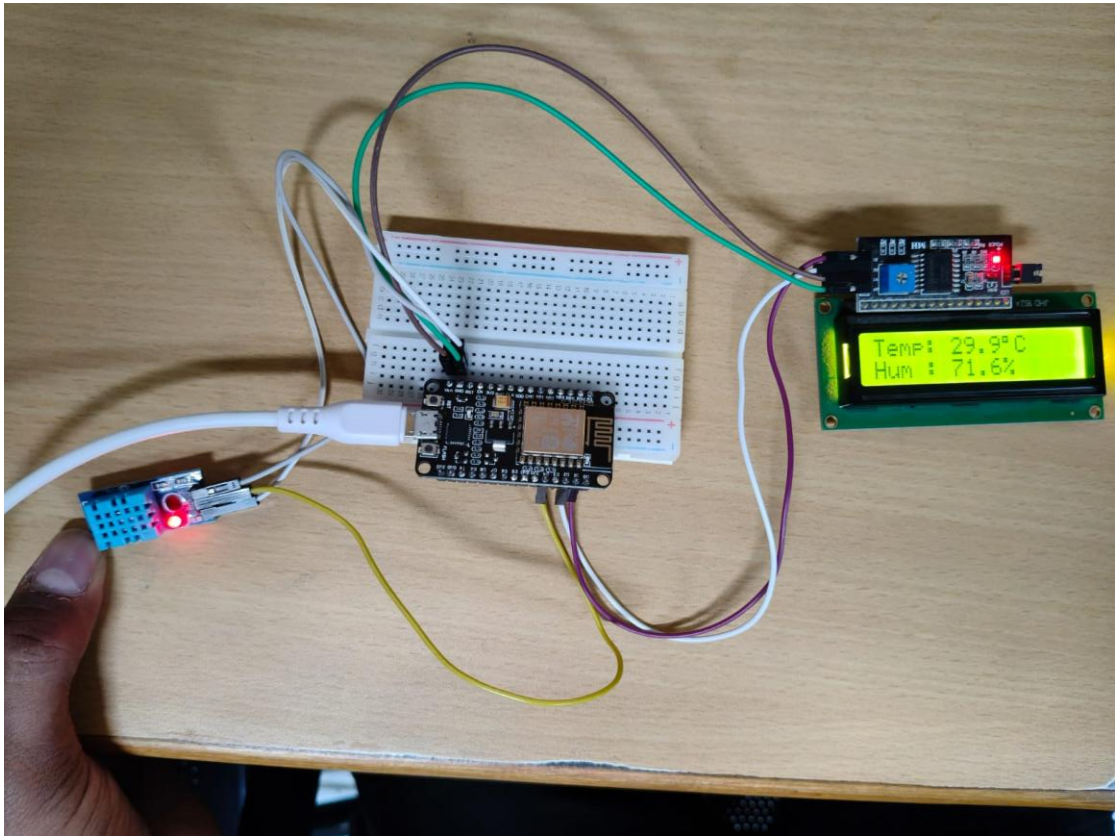
Working sequence

**Sensing → Processing → Local Display → MQTT
Transmission → Cloud Storage → Data Analysis**

10. Bill of Materials:

S.No.	Component	Quantity	Approx. Cost (₹)
1.	ESP8266 NodeMCU	1	250–400
2.	DHT11 Sensor	1	50–100
3.	16×2 I2C LCD	1	120–200
4.	Breadboard	1	80–150
5.	Jumper Wires	1 set	50–100
6.	USB Cable	1	50–100

11. Prototype Implementation:



The prototype was implemented using an ESP8266 Node MCU, DHT11 sensor and 16×2 I2C LCD.

The DHT11 sensor was positioned to measure the surrounding environmental conditions. The sensor was connected to the ESP8266, while the LCD was connected through the I2C interface.

After programming the ESP8266, the system was powered through USB. The ESP8266 established a Wi-Fi connection and transmitted the sensor data to Adafruit using MQTT.

The LCD provided local information, while Adafruit provided remote monitoring and graphical representation of the collected data.

Prototype stages

Stage 1: Hardware assembly

Stage 2: Sensor testing

Stage 3: LCD testing

Stage 4: ESP8266 Wi-Fi connection

Stage 5: MQTT configuration

Stage 6: Adafruit integration

Stage 7: Complete system testing

12. Results and Performance Analysis:

Performance

The system provides:

- Real-time temperature monitoring
- Humidity monitoring
- Local LCD display
- Wireless data transmission

- Cloud-based data storage
- Historical data visualization
- Remote monitoring capability

Advantages

1. Low-cost implementation.
2. Simple circuit design.
3. Wireless communication.
4. Remote monitoring.
5. Cloud-based data storage.
6. Easy visualization of sensor data.
7. Can be expanded for automatic HVAC control.

Limitations

1. DHT11 has limited measurement accuracy compared with industrial HVAC sensors.
2. The system depends on Wi-Fi availability.
3. The prototype monitors environmental parameters but does not directly control HVAC equipment.
4. MQTT/cloud communication depends on Internet connectivity.